

Inteligencia Artificial

Actividad Académicamente Dirigida

Luis Airam Saavedra Paiseo

Mayo-

Junio 2026

Índice

Contenido

Índice	1
1. Introducción	2
2. Prompt 0 – Baseline del Enunciado	2
2.1 Información disponible para el LLM y comportamiento	2
3. Prompt 1 – Mapa visual y posición del rival	2
3.1 Cambios y comportamiento	3
4. Prompt 2 – Adyacencias explícitas e historial de posiciones	3
4.1 Cambios respecto al prompt 1 y comportamiento	4
5. Prompt 3 – Versión final con pre-procesamiento en Java	4
5.1 Prompt y comportamiento observado	4
6. Alucinaciones y movimientos ilegales	5
7. Análisis entre versiones y legalidad vs alucinaciones	6
7.1 El problema de las alucinaciones en un entorno adversarial	7
8. Conclusiones	7


```
#aaaA      #
#           #
#           #
#      Bbbbb #
#####

TÚ: fila 5, col 9 | META: col 1 hacia Oeste (O) | Distancia: 8
RIVAL: fila 2, col 4 | Distancia a su meta: 11

Movimientos válidos: [N, S, O]
Analiza y responde. Última línea: ACCION: X
```

Texto del prompt 1.

3.1 Cambios y comportamiento

Se han añadido los siguientes cambios respecto a la versión inicial:

- Se añade un mapa visual completo: la cuadrícula ASCII con '#', 'a', 'b', 'A', 'B'
- Se le informa de la posición del rival y su distancia a su propia meta.
- Se indica la distancia propia a la meta para que el LLM compare entre ambas.

Con este prompt la mejora es notoria. El LLM ya no propone ir en direcciones en las que observe muros visibles en el mapa, ya que ahora los puede ver directamente. También empieza a mencionar al rival en su razonamiento y a valorar si va con ventaja o desventaja en el momento.

A pesar de estas mejoras, el agente sigue muriendo en tramos medios porque no detecta callejones, metiéndose en zonas con pocas salidas sin valorar el margen de pasos para poder salir antes de encerrarse.

4. Prompt 2 – Adyacencias explícitas e historial de posiciones

El prompt 1 mejora la percepción global pero el LLM aún no tiene información local de utilidad, no sabe con exactitud que hay en las celdas adyacentes ni recuerda el camino que ha seguido, de manera que tiende a dar vueltas en la misma zona o meterse en pasillos sin salida.

Este prompt añade una descripción matemática de cada celda vecina y las últimas 3 posiciones visitadas.

```
ADYACENCIAS:

Norte (N): libre
Sur (S): RASTRO-prohibido
Este (E): MURO-prohibido
Oeste(O): libre

HISTORIAL (antigua → reciente):

fila 5, col 11 → fila 5, col 10 → fila 5, col 9

PUEDES ELEGIR: [N, O]
```

Texto del prompt 2

4.1 Cambios respecto al prompt 1 y comportamiento

Se han añadido los siguientes cambios respecto al prompt anterior:

- Adyacencias: para cada una de las 4 direcciones se indica el tipo de celda (muro, libre, jugador) y si es permitida o no.
- Se añade el historial de las 3 últimas posiciones visitadas en orden cronológico.

El agente deja de volver sobre sus pasos gracias al historial, impidiendo que recaiga en zonas ya visitadas. Las adyacencias eliminan casi al completo la confusión sobre que celdas son accesibles y cuales no.

De esta forma solo queda resolver un problema, que evalúe a más de un paso de distancia, evaluando así si el siguiente movimiento le lleva a una zona con muchas salidas o a un pasillo sin retorno por ejemplo.

5. Prompt 3 – Versión final con pre-procesamiento en Java

El límite con los prompts anteriores era que le pedíamos al LLM que realizase tanto el análisis táctico como la decisión final. Un LLM es bueno para razonar en lenguaje natural, pero carece de eficiencia calculando heurísticas numéricas. Por tanto se han dividido responsabilidades; Java hace el análisis cuantitativo y el LLM aplica el razonamiento cualitativo sobre datos ya procesados. Llegando así a este enfoque de pre-procesamiento táctico, antes de construir el prompt, Java extrae las variables del estado del juego y las inyecta al LLM como datos ya interpretados, así este último está listo para razonar y no tiene ya datos crudos como anteriormente.

Se han utilizado las siguientes heurísticas y/o estrategias para el pre-procesamiento de datos:

- Distancia propia a la meta y distancia del rival a su meta.
- Salidas disponibles por cada movimiento válido: cuantas celdas libres hay desde la casilla de destino.
- Detección de callejones, evitando movimientos que llevan a celdas con 0 salidas libres, marcándolos como PROHIBIDOS y eliminándolos de la lista de candidatos a enviar en el prompt.
- Lista de movimientos que reducen la distancia a la meta (acercanAMeta)
- Decisión estratégica automática: Java elige entre AVANZAR A META o ENCERRAR AL RIVAL según la ventaja relativa y distancia propia.

Java aplica la siguiente tabla de decisión antes de construir el prompt, de forma que ya le indica al LLM que se ha decidido hacer y él solo tiene que tomar la decisión de hacia donde moverse:

Condición	Estrategia
Distancia propia ≤ 3	AVANZAR A META (sprint final)
Ventaja propia ≥ 3 columnas	ENCERRAR AL RIVAL
En igualdad o ventaja leve	AVANZAR A META
En desventaja	AVANZAR A META (urgente)

Tabla de decisión pre-procesada por Java

5.1 Prompt y comportamiento observado

El prompt utilizado para esta última versión ha sido el siguiente:

Eres una moto de luz en Tron. N=arriba, S=abajo, E=derecha(col+1), O=izquierda(col-1).

No puedes pisar muros(#), rastros(a/b) ni al rival. Ganas llegando a tu columna meta.

MAPA (tu=B rastro=b, rival=A rastro=a):

```
#####  
#a                #  
#aaaA            #  
#                #  
#                #  
# #Bbbbbbbbbb    #  
#####
```

TU: fila 5, col 1 | META: col 13 hacia Este (E) | Distancia: 12

RIVAL: fila 2, col 4 | Distancia a su meta: 10

HISTORIAL (antigua a reciente): fila 5, col 13 -> fila 5, col 12 -> fila 5, col 2

ADYACENCIAS: Norte(N):libre Sur(S):MURO-prohibido Este(E):libre Oeste(O):MURO-prohibido

SALIDAS: N:3sal E:4sal

PUEDES ELEGIR: [N, E]

MOVIMIENTOS QUE TE ACERCAN A TU META: [E]

=== ESTRATEGIA DE ESTE TURNO: AVANZAR A META ===

Vas igual o más cerca de tu meta (columna 13). Prioriza avanzar hacia Este (E).

Razona en 2 líneas aplicando la estrategia indicada.

Última línea OBLIGATORIAMENTE: ACCION: X (X de: [N, E])

Texto del prompt final

Y su comportamiento observado es que ha conseguido vencer al MiniMax en la primera ejecución, completando la partida en 12 ciclos solamente. El agente ya no se queda encerrado gracias a que Java elimina todos los callejones que pudiesen llevar al LLM a esta situación antes de que tenga que elegir. El razonamiento se mantiene coherente en todos los turnos: aplica la estrategia inyectada sin divagaciones.

6. Alucinaciones y movimientos ilegales

En las primeras versiones se observa un alucinamiento que poco a poco va apareciendo menos hasta desaparecer por completo, en la versión base el LLM menciona moverse “al Oeste” por ejemplo y el Oeste está o bien bloqueado por rastro propio o bien por un muro, aunque la lista de movimientos válidos lo excluye, Java fuerza la primera acción válida de la lista, generando un comportamiento errático.

Un ejemplo de esto ha sido que el LLM razona que el Oeste está libre en situaciones en las que el prompt le indica que Oeste no está libre, contradiciendo el texto del razonamiento a los datos proporcionados.

Al pasar al primer prompt creado a partir del base las alucinaciones se reducen significativamente. El LLM raramente vuelve a proponer una acción que no esté en la lista de movimientos válidos ya que tiene constancia del mapa y puede confirmar visualmente si es válida o no. Pero ocasionalmente falla en tramos con rastros densos porque llega a confundir el carácter ‘a’ con el espacio libre, pero son casos muy aislados.

Siguiendo con el segundo prompt, aquí las alucinaciones son mínimas. En las ejecuciones de prueba solo se detectó un caso en el que el LLM intentó razonar sobre una dirección marcada como prohibida, pero en la conclusión final (en la línea ACCION:) acaba siempre devolviendo una acción legal. Tener un historial de acciones parece conseguir anclar el razonamiento del modelo a datos concretos, evitando que divague.

En el último prompt se consiguen eliminar por completo las alucinaciones. Al eliminar los callejones en Java y proporcionar exactamente la lista de direcciones candidatas, el espacio de respuesta del LLM queda reducido a responder únicamente lo viable y válido. El modelo ya no menciona movimientos fuera de la lista de permitidos. Puede contener impresiones menores como referirse a columnas con un pequeño error de conteo, pero la decisión final es siempre correcta.

MiniMax garantiza la legalidad por construcción del árbol de búsqueda, el LLM garantiza lo mismo porque Java filtra el espacio antes de que él decida.

7. Análisis entre versiones y legalidad vs alucinaciones

Versión		Técnicas añadidas	Comportamiento observado	Alucinaciones
Prompt (Baseline)	0	Solo posición propia, columna meta y movimientos válidos	Se mueve hacia la meta en línea recta sin leer el entorno. Falla al primer obstáculo.	Sí — propone moverse al Oeste cuando el Oeste está bloqueado
Prompt (+Mapa +Rival)	1	Mapa visual ASCII, posición y distancia del rival	Ya no ignora muros visibles. Sigue sin detectar callejones ni analizar espacio libre.	Reducidas — pero aún intenta moverse a celdas bloqueadas por rastros en tramos finales
Prompt (+Adyacencias +Historial)	2	Descripción celda por celda, historial de 3 posiciones	Evita volver sobre sus pasos. Detecta muros adyacentes sin alucinaciones. Aún puede quedar encerrado.	Muy bajas — solo ocasionalmente ignora el historial
Prompt (Final — Java+LLM)	3	Pre-procesamiento Java: callejones, ventaja relativa,	Gana al Minimax en 12 ciclos. Nunca mueve a	Ninguna — Java filtra ilegales antes

Versión	Técnicas añadidas	Comportamiento observado	Alucinaciones
	movimientos que acercan a meta, estrategia inyectada	celda ilegal. El LLM solo razona y confirma.	de enviar el prompt

Tabla comparativa de versiones

7.1 El problema de las alucinaciones en un entorno adversarial

Un LLM puede generar texto coherente que no corresponde a la realidad del estado de juego. En este caso, una alucinación tiene consecuencias directas, si por ejemplo el modelo propone moverse a una dirección ilegal, Java la intercepta y fuerza retroceder, forzando la pérdida del turno estratégicamente.

Se han observado diversas alucinaciones, como, por ejemplo:

- Contradicción con la lista de válidos: el LLM razona sobre una dirección bloqueada que no está en las válidas. Ocurre en el prompt 0 mayormente y en algún caso ocasional en el prompt 1.
- Confusión de caracteres en el mapa: el LLM interpreta 'a' (su propio rastro) como espacio libre. Detectado puntualmente en el prompt 1.
- Error de referencia espacial: el LLM indica por ejemplo " me muevo al Norte para acercarme a la columna X", confundiendo filas con columnas. No produce movimientos ilegales, pero muestra que no tiene representación espacial nativa.

8. Conclusiones

La evolución de prompts de esta práctica ilustra un principio general del diseño de sistemas basados en LLM: el modelo más fiable es aquel que menos tiene que calcular por sí mismo.

Cada iteración del prompt mejoró el rendimiento del agente sin añadir más información al azar, sino reduciendo la ambigüedad y pre-procesando el análisis cuantitativo en el lado de Java.

Datos a puntualizar como importantes:

El mapa visual ayuda, pero no es suficiente por sí solo. El LLM puede leer el mapa, pero no tiene una representación espacial real, pudiendo llevarle a confundir caracteres o errores de conteo.

La descripción explícita de adyacencias reduce alucinaciones más que el mapa completo. Tener información local precisa es más efectivo que tener información local ambigua.

El historial de posiciones ancla el razonamiento del modelo evitando bucles. Sin esto, el LLM tiende a tomar siempre las mismas decisiones en estados similares sin detectar que está entrando en un ciclo.

El pre-procesamiento ha resultado ser la técnica más efectiva. Java al ser determinista y tener acceso al estado exacto del juego le proporciona al LLM lo justo y necesario para razonar la información precisa y válida, llevándolo a tomar las mejores decisiones en consonancia de utilizar ambos.

La legalidad no es algo garantizado solo pidiéndolo al modelo, se garantiza filtrando todo el espacio de decisión antes si quiera de que éste llegue a intervenir.